Training on Traffic Without Retaining It

Privacy and Efficiency Are the Same Constraint for a Continually Trained Router

Zen LM Research, Hanzo AI
research@hanzo.ai

2026

Abstract

A router that learns from production traffic needs supervision from the requests people actually send. The obvious way to get it—store the prompts—is a privacy failure. The usual mitigations are worse than they look: token identifiers are a lossless encoding of the text they came from, and frozen-backbone embeddings are invertible well enough that published attacks recover most short inputs verbatim. Both are routinely described as anonymization. Neither is.

We argue for the stronger and simpler position: *do not retain*. What training needs from a request is a gradient, and the request is only the courier that carries it. Once the gradient is applied, keeping the courier adds no capability and all of the risk. We describe a substrate that computes the update online and persists only model weights and aggregates, and a two-tier consent model separating what trains *your* router from what trains *everyone's*.

The argument's force is that it costs nothing. Retention is not a capability we give up to buy privacy: the low-dimensional features a router actually needs are cheaper to compute, cheaper to store, and—on the evidence of the router already serving our traffic— sufficient. Privacy and efficiency here are not a trade-off to balance but the same constraint seen twice.

We include an audit of our own system, which fails part of this standard today: our routing ledger persists a frozen-backbone embedding under a code comment asserting it is "never prompt text." That claim is true and misleading, and we treat finding it as the point rather than an embarrassment to omit. A privacy argument that has not been run against its author's own code is a marketing claim.

1 The Problem

A model that routes requests must learn what a good routing decision looks like, and the only honest source of that knowledge is the traffic itself: which requests were hard, which were trivial, which needed vision, which needed audio. Synthetic difficulty labels teach a model to imitate the heuristic that generated them. Real traffic is the ground truth, and it belongs to the people who sent it.

This puts an ordinary product in an uncomfortable place. The data that makes the system good is the data it has the least right to keep. The industry's answer has largely been to keep it anyway, under a word—"anonymized"—doing more work than it can bear.

2 Two Mitigations That Do Not Work

2.1 Token identifiers are the text

Storing token IDs rather than strings is sometimes offered as a privacy measure. It is not one. Tokenization is a bijection with a published dictionary: $\text{detokenize}(\text{tokenize}(x)) = x$, exactly, for every x. A token sequence is the prompt in a different alphabet, and the key is a public

download. There is no threat model in which storing token IDs is meaningfully different from storing the prompt, because it is not different from storing the prompt.

2.2 Embeddings invert

The subtler mitigation is to store an embedding: a fixed-width vector from a frozen backbone, which "is not the text." This is where the reasoning usually stops, and it should not.

An embedding is trained to preserve meaning. Preserving meaning is exactly the property that makes it reconstructable, and the reconstruction is not hypothetical. Inversion attacks against text embeddings recover a large fraction of short inputs *verbatim* from the vector alone [1], treating inversion as iterative refinement in embedding space and needing no access to the encoder's weights. Later work extends the result across encoders and to longer inputs.

The security posture of "we only store embeddings" is therefore roughly the security posture of "we only store the prompt, lightly encrypted with a key we published." The vector is smaller and harder to read by eye. It is not private.

A useful way to hold the distinction: an embedding is lossy but *semantically faithful*, and faithfulness is what inverts. Low-dimensional, purpose-built features— length, task class, modality, observed latency—are lossy in the way that matters. They discard the content and keep a decision. Nothing inverts a scalar.

3 The Principle

> *The gradient is what training wants. The request is the courier. Do not keep the courier.*

Once an update has been applied, the request has already given the system everything it had to give. Retaining it buys no additional learning; it only creates an asset that can be breached, subpoenaed, correlated, or quietly repurposed by a future version of ourselves with different judgment. Deletion policies mitigate that risk. Never writing eliminates it. The two are not close: a retention window is a promise, and an absent column is a fact.

This reframes the design question. Rather than asking how strongly to anonymize what we store, ask what the smallest thing is that produces the same gradient, and whether it must be stored at all. For a router, the answer is: less than you would guess, and no.

4 The Substrate

4.1 Stream, update, drop

The event exists long enough to produce a gradient and is then discarded. What persists is model weights and aggregate metrics—quantities that are already the output of the training we are permitted to do.

One concession to reality: pure online SGD over an unshuffled stream trains badly, so a short-lived buffer (minutes) collects a batch before the update. The buffer is memory, not a table. The distinction we are drawing is not "fast deletion" versus "slow deletion" but *written* versus *not written*.

4.2 Features, not representations

The router already computes what it needs to route. If those features are low-dimensional and purpose-built, they are safe to hold and cheap to move. If they are a backbone embedding, they are the prompt by another name and must not be persisted (Section 2.2). This is one seam

and it should have one form, consumed by the live router and by any successor trained from its decisions.

4.3 Two-tier consent

Consent is not one bit, because the question is not one question.

Table 1: The two scopes. Conflating them is how "we train on your data" becomes true in a way the user did not agree to.

Scope	**What it trains**	**Default**
Your own head	your router, from your traffic	opt-*out*
The shared base	everyone's router	opt-*in*

An organization's own events tuning its own router is the service working as described, and defaults to on with an explicit way off. The same events joining a fit that ships to everyone else is a different act with a different risk, and defaults to off until asked. The two must not read the same flag with the same predicate, or one of them is wrong.

4.4 What is deliberately absent

No prompt text. No token identifiers. No backbone embeddings. No cross-org join key. Formal guarantees (DP-SGD and its relatives) are available if the shared base fit ever needs one, but they answer a question that arises only after you have decided to retain. Not retaining is the cheaper instrument, and it is available first.

5 Why This Costs Nothing

The argument would be weaker if it were a sacrifice. It is not.

Table 2: The same choice, read twice.

Decision	**Privacy reading**	**Efficiency reading**
Features, not embeddings	does not invert	smaller model; no backbone pass
Do not retain	nothing to breach	no store, no lifecycle, no cost
Stream the update	no corpus assembled	no offline pipeline
Own head first	data stays in its scope	personalization beats a global fit

Privacy and efficiency are usually in tension because privacy is usually bolted on: you buy it with encryption, with auditing, with retention machinery, all of which cost. Here the private choice and the cheap choice are the same choice, because both follow from asking what the system actually needs and declining to acquire more. The alignment is not a happy accident. Minimality is one property, and it is legible from either side.

There is a modeling claim inside this and we state it as a claim: a router that consumes purpose-built features may be *better* than one that reads raw tokens through a large encoder, not merely cheaper and safer. Our production router routes across every modality we serve on such features today. Whether that generalizes to expert-granularity routing is untested, and Section 7 says so.

6 Discovering the Latent Space

A routed architecture that spans model families needs a shared representation for its experts to interoperate through—a latent space between models and modalities. Our own architecture papers specify a projection into that space and, in doing so, assume four things without measuring any of them: which layers to tap, which models are worth fusing, what the shared dimension is, and that a projection suffices at all.

Continual training turns those assumptions into measurements. A system that routes across every modality and records where performance actually comes from is an instrument, and what it measures is precisely which models and which layers carry signal worth aligning. The latent space is then *discovered* rather than designed—and discovered from aggregates, which is the form we are permitted to keep. That the privacy-preserving artifact and the architecturally informative artifact are the same object is the previous section's point appearing a third time.

7 Audit: Where We Fail This Today

A privacy argument not run against its author's own system is a marketing claim. Ours, audited at the time of writing:

Table 3: Our own compliance with the above.

Property	**Status**	**Note**
No prompt text persisted	**Holds**	bodies are opt-in, redacted by default
Two-tier consent	**Holds**	own head opt-out; shared base opt-in
Nullable reward	**Holds**	unscored $\neq$ zero; dismissal records nothing
Counterfactual logged	**Holds**	the shadow arm, for off-policy learning
No embeddings persisted	**FAILS**	the ledger stores a frozen-backbone embedding
Do not retain	**FAILS**	events are written, not streamed
Consent read consistently	**Unclear**	one flag, two predicates (Section 4.3)

The first failure is the instructive one. Our routing ledger carries a feature column whose contract describes it as "the optional frozen-backbone embedding… passed through to the routing ledger for training (never prompt text)." Every word is accurate. The parenthetical is nonetheless the sentence a reader will rely on, and it is false comfort: by Section 2.2, a frozen-backbone embedding is the prompt in a form that inverts. The comment is not a lie; it is a true statement doing a false job, which is the more common failure and the harder one to catch. We caught it by asking what the vector was, not by reading the assurance next to it.

The remedy follows from Section 4: the column stops being an embedding, and stops being persisted. Both, not either.

We report this because a paper claiming a trustworthy substrate, published by an organization that had not checked, would be evidence of nothing. The check is the contribution. The fix is scheduled work, and this paper will be wrong about our own status the moment it lands—which is the correct direction for a paper like this to age.

8 What Is Not Established

- The streaming substrate is specified here, not deployed. Sections 4 and 7 must be read together.
- That features suffice for expert-granularity routing is a hypothesis. It is supported by a production router that routes at *model* granularity on features, which is a weaker claim

than the one we need.

- Non-retention is an architectural guarantee, not a formal one. It resists breach and subpoena because the data is absent. It does not by itself bound what a trained model memorizes, which is a different question with different instruments [2].
- We publish no numbers. There is no evaluation of the substrate because it does not exist yet.

9 Related Work

Embedding inversion is the load-bearing citation: Morris et al. [1] show text embeddings can be inverted to recover most short inputs exactly, which is what disqualifies the "we only store vectors" position. Differentially private training [2] bounds what a released model reveals about any single training example—complementary to, not a substitute for, declining to retain the example. Federated learning [3] shares the instinct that the update should travel rather than the data; our setting is simpler, since the data is already at the server that must learn from it, and the discipline is to not write it down. Contextual bandit logging [4] gives the shape of the record—context, action, reward, propensity—and the reason to log the counterfactual arm.

Our architecture line is specified separately: MoDE [5] for the harvested-expert model family, and MUEN [6] for the router line whose v2 supplies the traffic this paper is about.

Availability

Router, ledger, and consent implementation: the `ai` service (private). Architecture: https://github.com/hanzoai/mode, https://github.com/hanzoai/muen.

References

[1] J. X. Morris, V. Kuleshov, V. Shmatikov, and A. M. Rush. *Text Embeddings Reveal (Almost) As Much As Text*. EMNLP 2023. arXiv:2310.06816. https://arxiv.org/abs/2310.06816

[2] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. *Deep Learning with Differential Privacy*. CCS 2016. arXiv:1607.00133. https://arxiv.org/abs/1607.00133

[3] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS 2017. arXiv:1602.05629. https://arxiv.org/abs/1602.05629

[4] L. Li, W. Chu, J. Langford, and R. E. Schapire. *A Contextual-Bandit Approach to Personalized News Article Recommendation*. WWW 2010. arXiv:1003.0146. https://arxiv.org/abs/1003.0146

[5] Zen LM Research, Hanzo AI. *MoDE v3: Mixture of Diverse Experts*. https://github.com/hanzoai/mode

[6] Zen LM Research, Hanzo AI. *MUEN v3: Multimodal Mixture of Unbound Experts*. https://github.com/hanzoai/muen